

NYC Taxi 2025 Data Cleaning Logs

Professional cleaning summary for the PySpark based NYC Yellow Taxi Parquet project

Project	NYC Yellow Taxi 2025 Data Cleaning
Notebook	NYC_Taxi_Cleaned_data.ipynb
Repository Path	SummerDrive/Cohort-1/NYC Taxi/Data Cleaning/Cleaned Dataset
Processing Tool	PySpark
Input Format	12 monthly Parquet files
Output Format	Cleaned Parquet dataset and small CSV summaries/samples
Prepared For	Data Analysis and Dashboarding
Prepared By	Prateek Shakya
Date	29 June 2026

Purpose

This document records the data cleaning work completed on the NYC Yellow Taxi 2025 raw Parquet dataset. The aim was to prepare a reliable, analysis-ready dataset by removing invalid trips, fixing data types, creating useful time and fare features, and saving the cleaned output in a format suitable for large-scale analysis.

Important Note

The cleaning was designed for PySpark because the dataset contains multiple monthly Parquet files and can be too large for normal Pandas processing. Spark is used for the heavy cleaning work, while small summaries can later be exported for dashboards.

1. Cleaning Workflow Followed

The project follows a simple big-data workflow: start Spark, read Parquet files, inspect the data, clean invalid records, engineer new columns, validate the result, and save the cleaned output.

Stage	Action	Reason
Setup	Installed PySpark and started a SparkSession using local cores.	Creates the Spark engine required to process large Parquet files.
Read	Loaded one month first, then loaded all 12 monthly files with mergeSchema.	Testing one file first avoids confusion, and mergeSchema handles small schema differences between months.
Inspect	Checked schema, sample rows, row count, column count, missing values, and summary statistics.	Understanding the raw data before cleaning prevents random or unsafe transformations.
Clean	Filtered invalid fares, distances, trip durations, location IDs, payment codes, tips, and unrealistic speeds.	Removes records that would distort analysis results.
Feature Engineering	Created columns such as trip_duration_minutes, pickup_hour, pickup_day, pickup_month, speed_mph, fare_per_mile, tip_percentage, and is_cbd.	Makes the dataset more useful for trend, demand, fare, tip, and congestion analysis.
Save	Saved cleaned data as Parquet and small summaries as CSV.	Parquet is efficient for large data; CSV is easier for dashboard tools and GitHub preview.

2. Environment and Setup

Item	Used In Project
Language	Python
Main Library	PySpark
Execution Platform	Google Colab
Spark App Name	nyc-yellow-taxi-2025
Master	local[*] - use all available CPU cores
Shuffle Partitions	16 for local execution
Timezone	America/New_York
Raw Data Format	Parquet

3. Detailed Cleaning Log

The table below explains each cleaning action in human language. It also records the PySpark method used and the reason behind the step.

No	Cleaning Step	What Was Done	Why It Was Needed	PySpark Method
1	Duplicate rows	Removed exact duplicate trip records.	Duplicate trips can inflate trip counts, revenue, and demand analysis.	dropDuplicates()
2	Datetime columns	Checked pickup and dropoff timestamp fields.	Trip duration and time-based analysis require valid timestamp columns.	printSchema(), to_timestamp
3	Missing essential times	Removed rows with missing pickup or dropoff datetime.	Without these fields, trip duration cannot be calculated correctly.	na.drop(subset=...)
4	Dataset year	Kept only trips where pickup year is 2025.	The project focuses only on NYC Taxi 2025 records.	filter(year(...) == 2025)
5	Trip duration	Created trip_duration_minutes and removed trips with duration <= 0 or very long durations.	Negative or zero duration is impossible, and very long durations are likely outliers.	withColumn(), filter()
6	Trip distance	Removed trips with distance <= 0 and unrealistic high distances.	Zero or negative distance trips do not represent normal taxi rides.	filter()
7	Passenger count	Converted passenger count to integer and kept realistic values, generally 1 to 6.	Invalid passenger values can affect demand and occupancy analysis.	cast('int'), filter()
8	Fare amount	Removed negative, zero, and extremely high fare values.	Fare amount is central to revenue analysis, so it must be positive and reasonable.	filter(F.col('fare_amount') > 0)
9	Total amount	Removed invalid total amount values.	Total amount is used for revenue, so wrong values can distort dashboard results.	filter()
10	Tip amount	Removed records with negative tips.	Tips should not be negative for normal fare analysis.	filter(F.col('tip_amount') >= 0)
11	Location IDs	Kept pickup and dropoff location IDs in the valid NYC taxi zone range.	Invalid zone IDs break location analysis and zone lookup joins.	between(1, 265)
12	Payment type	Kept only known payment type codes.	Payment codes are needed for cash, card, and tip behavior analysis.	isin([...])
13	Speed check	Created speed_mph and removed unrealistic speeds.	Unrealistic speed usually means incorrect duration, distance, or timestamp data.	withColumn(), filter()
14	Time features	Created pickup_date, pickup_hour, pickup_day, and pickup_month.	These fields support monthly trends, hourly demand, and day-wise analysis.	to_date(), hour(), date_format()
15	Fare features	Created fare_per_mile and tip_percentage with safe division logic.	These features help compare trip economics across time and location.	withColumn(), when().otherwise()
16	CBD flag	Created is_cbd flag using cbd_congestion_fee where available.	This supports congestion-fee and CBD impact analysis.	when(F.col('cbd_congestion_fee') > 0, 1)

4. New Analysis Columns Created

Column	Meaning	Used For
trip_duration_minutes	Trip time in minutes from pickup to dropoff.	Duration analysis and invalid trip detection.
speed_mph	Average speed calculated from distance and duration.	Outlier detection and trip quality checks.
pickup_date	Date of pickup.	Daily trend analysis.
pickup_hour	Hour of pickup from 0 to 23.	Hourly demand and peak hour analysis.
pickup_day	Day name of pickup.	Weekday and weekend patterns.
pickup_month	Month number or month name depending on notebook implementation.	Monthly trend and partitioned output.
fare_per_mile	Fare amount divided by trip distance.	Fare efficiency and outlier analysis.
tip_percentage	Tip amount as a percentage of fare amount.	Tip behavior analysis.
is_cbd	Flag for trips with CBD congestion fee.	Congestion-fee impact analysis.

5. Final Validation Checklist

Check	Expected Result
Schema review	All important columns have correct data types.
Missing values	Essential columns such as pickup time, dropoff time, fare, distance, and location IDs should not contain invalid nulls.
Duplicate records	Duplicate trip rows should be removed.
Fare and amount	fare_amount and total_amount should be positive and reasonable.
Distance	trip_distance should be positive and within a realistic limit.
Duration	trip_duration_minutes should be positive and within a realistic limit.
Speed	speed_mph should be realistic after filtering.
Year	pickup year should be 2025.
Output	Cleaned Parquet and small CSV summary/sample should be saved successfully.

6. Output Files

Output	Purpose
cleaned_nyc_taxi_2025_parquet	Main cleaned dataset saved in Parquet format for efficient big-data reading.
cleaned_nyc_taxi_2025_sample_csv	Small CSV sample for quick preview, GitHub upload, or dashboard testing.
summary CSV files	Small aggregated outputs for dashboards such as monthly revenue, hourly demand, fare trends, and tip analysis.

7. Good Practices Followed

- Used PySpark for heavy processing instead of pulling the full dataset into Pandas.
- Loaded and inspected one month first before combining all monthly files.
- Used filters to remove impossible or unrealistic trip records.
- Used withColumn for feature engineering instead of modifying data outside Spark.
- Avoided collect() and toPandas() on the full raw dataset.
- Saved the large cleaned output as Parquet and kept CSV only for small summaries or samples.
- Used overwrite mode so the notebook can be safely rerun without creating duplicate output folders.

8. Cleaning Summary

The raw NYC Taxi 2025 Parquet files were processed using PySpark. The cleaning removed invalid trips, corrected key data types, filtered unrealistic values, and created new analytical columns. The final cleaned dataset is suitable for exploratory data analysis, dashboards, and further business questions such as demand trends, fare analysis, tip behavior, zone performance, and congestion-fee study.

9. Reference Used

Learning PySpark by Building It - NYC Yellow Taxi 2025 project handbook, June 2026. The workflow follows the handbook structure: setup SparkSession, read Parquet, inspect data, filter rows, build new columns, handle dates, group/summarise, and write Parquet/CSV outputs.

End of Cleaning Log